70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · MT3

Sun'iy intellektga asoslangan zamonaviy kompyuter tizimlari arxitekturalarini (CPU, GPU, TPU, NPU va FPGA) qiyosiy tahlil qilish va ularning AI ilovalaridagi samaradorligini baholash

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/MT3
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/MT3
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu mustaqil ishda siz besh xil hisoblash arxitekturasini analitik modellar va simulyatsiyalar orqali solishtirasiz: rasmiy cho'qqi qiymatlari bilan Roofline modeli, TPU sistolik massivining takt darajasidagi simulyatori, FPGA quvuri va fixed-point arifmetikasi, NPU uchun INT8 kvantlash, energiya va TCO kalkulyatori hamda yakuniy ko'p mezonli tanlov matritsasi. Real tezlatkichlar (GPU, TPU, NPU, FPGA) sandbox'da yo'q, shuning uchun ular modellar bilan baholanadi; yagona real o'lchov qurilmasi — 2 yadroli CPU, uning natijalari boshqa arxitekturalarga ekstrapolyatsiya qilinadi. Jadvaldagi barcha narx, quvvat va unumdorlik raqamlari taxminiy.

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/MT3`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → MT3 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. Arxitekturalar pasporti va Roofline modeli
2. 2-qadam. TPU sistolik massivi: takt darajasidagi simulyator va MXU foydalanish modeli
3. 3-qadam. FPGA: quvur (pipeline), II va fixed-point aniqlik simulyatsiyasi
4. 4-qadam. NPU uchun INT8 kvantlash: aniqlik, hajm, kechikish va grafik bo'linishi
5. 5-qadam. Real CPU'da GEMM va konvolyutsiya o'lchovlari, boshqa arxitekturalarga ekstrapolyatsiya
6. 6-qadam. Energiya samaradorligi (TOP/s/Vt, mJ/tasvir) va 3 yillik TCO
7. 7-qadam. Yakuniy tanlov matritsasi: WSM, TOPSIS va vaznlar sezgirligi
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. Arxitekturalar pasporti va Roofline modeli

MAQSAD

CPU (AMD EPYC 9654), GPU (NVIDIA H100 SXM5), TPU (Google TPU v4), NPU (Hailo-8), chekka SoC (Jetson AGX Orin) va FPGA (AMD Alveo U250) uchun ochiq spetsifikatsiyalardagi cho'qqi qiymatlar (INT8/FP16 TOP/s, xotira o'tkazuvchanligi, quvvat) jadvali tuziladi. Har biri uchun ridge nuqtasi (cho'qqi / o'tkazuvchanlik) va TOP/s/Vt hisoblanadi. Keyin 6 ta tipik AI yadrosi (katta GEMM, ResNet conv3x3, BERT FFN, LLM dekod GEMV, depthwise conv, ReLU) uchun arifmetik intensivlik formuladan topiladi va Roofline bo'yicha erishiladigan unumdorlik hamda chegara turi (hisoblash yoki xotira) ko'rsatiladi. Log-log Roofline grafigi chiziladi. Barcha cho'qqi qiymatlari taxminiy — ular avlod, chastota va rejimga qarab o'zgaradi.

> **Kutilgan natija:** Pasport jadvali, AI jadvali (GEMM ~2700, GEMV ~2 op/bayt), Roofline jadvali: katta GEMM hamma joyda H, GEMV/ReLU hamma joyda X; LLM dekod token/s ~ BW/7 GB; grafik.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. Arxitekturalar pasporti va Roofline modeli: CPU, GPU, TPU, NPU, FPGA
# Roofline: erishiladigan unumdorlik P = min(P_cho'qqi, AI * BW), bu yerda
#   AI (arifmetik intensivlik) = amallar soni / xotiradan o'qilgan-yozilgan baytlar (op/bayt).
# Cho'qqi qiymatlar ishlab chiqaruvchilarning ochiq spetsifikatsiyalaridan olingan, TAXMINIY
(zich, sparsity'siz).
import math
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option("display.width", 160)

# --- Qurilmalar pasporti: cho'qqi (Top/s), xotira o'tkazuvchanligi (GB/s), quvvat (Vt) ---
# INT8 ustuni — inferens uchun, FP16/BF16 — o'qitish uchun. Raqamlar taxminiy, avlod/takt
chastotasiga qarab o'zgaradi.
NAN = float("nan")   # ochiq spetsifikatsiyada berilmagan yoki asosiy rejim emas
QURILMA = [
    # nom,                          tur,    FP16/BF16 TFLOP/s, INT8 TOP/s, BW GB/s, quvvat Vt,
izoh
    ("AMD EPYC 9654 (96 yadro)",   "CPU",    7.4,    29.5,   460.8, 360, "FP32 ~7,4 TFLOP/s
(AVX-512, 2,4 GHz); BF16 alohida tezlanmaydi deb olindi"),
    ("NVIDIA H100 SXM5",           "GPU",  989.0,  1979.0,  3350.0, 700, "Tensor Core, zich;
HBM3"),
    ("Google TPU v4 (1 chip)",     "TPU",  275.0,   275.0,  1200.0, 200, "2 yadro x 4 MXU
128x128, 1,05 GHz; quvvat ~170-200 Vt"),
    ("Hailo-8 (M.2)",              "NPU",    NAN,    26.0,     4.0, 2.5, "tashqi DRAM yo'q,
xost bilan PCIe Gen3 x4 ~4 GB/s"),
    ("Jetson AGX Orin 64GB",       "SoC",    NAN,   138.0,   204.8,  60, "GPU+DLA; INT8 275
TOP/s sparsity bilan, zich ~138"),
    ("AMD Alveo U250",             "FPGA",   NAN,    33.3,    77.0, 225, "INT8 33,3 TOP/s, DDR4
4x"),
]
df = pd.DataFrame(QURILMA, columns=["nom", "tur", "fp16_tflops", "int8_tops", "bw_gbs", "vt",
"izoh"])
df["ridge_int8"] = df.int8_tops * 1e12 / (df.bw_gbs * 1e9)       # op/bayt: shundan yuqorida —
hisoblash chegarasi
df["tops_per_vt"] = df.int8_tops / df.vt
```

```
print("=== Arxitekturalar pasporti (taxminiy, ochiq spetsifikatsiyalar) ===")
print(df[["nom", "tur", "fp16_tflops", "int8_tops", "bw_gbs", "vt", "ridge_int8",
"tops_per_vt"]]
      .to_string(index=False, float_format=lambda v: f"{v:.1f}", na_rep="—"))
print("— : ochiq spetsifikatsiyada berilmagan yoki qurilmaning asosiy rejimi emas")

# --- AI ish yuklamalari: amallar va baytlar formulalardan hisoblanadi (INT8, 1 bayt/element) -
--
def gemm(M, K, N, b=1):
    ops = 2 * M * K * N
    bayt = b * (M * K + K * N + M * N)          # ideal: har matritsa xotiradan bir marta
o'tadi
    return ops, bayt

def conv(H, W, Ci, Co, k, b=1):
    ops = 2 * H * W * Ci * Co * k * k
    bayt = b * (H * W * Ci + k * k * Ci * Co + H * W * Co)
    return ops, bayt

def dwconv(H, W, C, k, b=1):
    ops = 2 * H * W * C * k * k
    bayt = b * (2 * H * W * C + k * k * C)
    return ops, bayt

# kalit: qisqa nom (jadval ustuni) ; qiymat: (to'liq nom, (amallar, baytlar))
YUKLAMA = {
    "GEMM":    ("GEMM 4096^3 (o'qitish bloki)",     gemm(4096, 4096, 4096)),
    "Conv3x3": ("ResNet conv3x3 56x56x64->64",      conv(56, 56, 64, 64, 3)),
    "BERT":    ("BERT FFN (128 token, 768->3072)",  gemm(128, 768, 3072)),
    "GEMV":    ("LLM dekod GEMV (1x4096x4096)",     gemm(1, 4096, 4096)),
    "DWconv":  ("Depthwise 3x3 112x112x32",         dwconv(112, 112, 32, 3)),
    "ReLU":    ("ReLU/qo'shish (element bo'yicha)", (112 * 112 * 64, 2 * 112 * 112 * 64)),
}
print("\n=== Yuklamalar: arifmetik intensivlik ===")
for k, (nom, (o, b)) in YUKLAMA.items():
    print(f"{nom:34s} amal={o / 1e9:9.3f} G  bayt={b / 1e6:8.2f} MB  AI={o / b:8.1f} op/bayt")

print("\n=== Roofline: erishiladigan INT8 unumdorlik, TOP/s (chegara turi) ===")
satrlar = []
for _, q in df.iterrows():
    satr = {"qurilma": q.nom.split(" (")[0]}
    for k, (nom, (o, b)) in YUKLAMA.items():
        ai = o / b
        p = min(q.int8_tops, ai * q.bw_gbs / 1e3)
        chegara = "H" if q.int8_tops <= ai * q.bw_gbs / 1e3 else "X"    # H - hisoblash, X -
xotira
        satr[k] = f"{p:8.2f} {chegara}"
    satrlar.append(satr)
print(pd.DataFrame(satrlar).to_string(index=False))
print("H = hisoblash chegarasi (compute-bound), X = xotira chegarasi (memory-bound)")
print("Eslatma: Hailo-8 uchun BW = xost PCIe — eng yomon holat; og'irliklar chip ichidagi
SRAM'ga sig'sa, real natija yuqoriroq.")

# --- LLM dekod misoli: tokenlar/s ~ BW / model hajmi (7B parametr, INT8 = 7 GB) ---
print("\n=== LLM dekod (7B, INT8, batch=1): token/s ~ BW / 7 GB ===")
for _, q in df.iterrows():
    tps = q.bw_gbs / 7.0
    print(f"{q.nom:28s} ~{tps:8.1f} token/s  (cho'qqi hisoblash ulushi: {100 * tps * 14e9 /
(q.int8_tops * 1e12):6.2f} %)")
print("Xulosa: dekod bosqichi deyarli hamma joyda xotira chegarasida — HBM o'tkazuvchanligi hal
qiladi.")

# --- Grafik: log-log Roofline ---
```

```
fig, ax = plt.subplots(figsize=(9, 5.5))
ai_q = [10 ** (e / 20) for e in range(-20, 101)]
for _, q in df.iterrows():
    ax.loglog(ai_q, [min(q.int8_tops, a * q.bw_gbs / 1e3) for a in ai_q], label=f"{q.tur}:
{q.nom.split(' (')[0]}")
for k, (nom, (o, b)) in YUKLAMA.items():
    ax.axvline(o / b, color="gray", ls=":", lw=0.8)
    ax.text(o / b, 0.02, k, rotation=90, fontsize=7, va="bottom")
ax.set_xlabel("Arifmetik intensivlik, op/bayt"); ax.set_ylabel("Erishiladigan INT8 TOP/s")
ax.set_title("Roofline: CPU / GPU / TPU / NPU / FPGA (taxminiy cho'qqilar)")
ax.set_ylim(0.01, 5000); ax.legend(fontsize=7, loc="upper left"); ax.grid(True, which="both",
alpha=0.3)
plt.tight_layout()
print("\nRidge nuqtasi qancha katta bo'lsa, qurilmani to'liq yuklash uchun shuncha 'qalin'
(qayta ishlatishli) yuklama kerak.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Arxitekturalar pasporti`, `=== Roofline: erishiladigan INT8`, `=== LLM dekod`

TAJRIBA QILIB KO‘RING

- Jadvalga NVIDIA A100 yoki L4 ni qo'shing (ma'lumot varag'idan qiymatlarni o'zingiz toping) va ridge nuqtasini solishtiring.
- BERT FFN da token sonini 128 dan 2048 ga oshiring — H100 qachon hisoblash chegarasiga o'tadi?
- Yuklamalarni FP16 (2 bayt/element) bilan qayta hisoblang: AI va chegara turi qanday o'zgaradi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. TPU sistolik massivi: takt darajasidagi simulyator va MXU foydalanish modeli

MAQSAD

NumPy'da output-stationary sistolik massiv takt-ma-takt simulyatsiya qilinadi: A satrlari chapdan kechikish (skew) bilan, B ustunlari yuqoridan oqib kiradi, registrlar har taktda siljiydi. Natija A@B bilan aynan mos kelishi, taktlar soni K+R+C-2 formulasiga tengligi va PE foydalanish ulushi tekshiriladi; t=6 dagi diagonal to'lqin fronti ko'rsatiladi. So'ngra TPU v4 ning 128x128 MXU (8 dona, 1,05 GHz = 275 TFLOP/s) uchun weight-stationary analitik model bilan real qatlamlarda (BERT, ResNet, LLM dekod, toq o'lcham) foydalanish va samarali TFLOP/s hisoblanadi.

> **Kutilgan natija:** Simulyatorda xato 0; K=256 da foydalanish ~90 %; MXU: BERT 4096 token ~94 %, GEMV M=1 ~0,4 %, toq N=130 ~45 %; grafik.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. TPU yuragi — sistolik massiv: takt darajasidagi simulyator va MXU 128x128
foydalanish modeli
# Chap tomondan A satrlari (kechikish bilan, "skew"), yuqoridan B ustunlari oqib kiradi; har PE
(protsessor elementi)
# har taktda bitta MAC bajaradi va ma'lumotni qo'shnisiga uzatadi. Natija PE ichida to'planadi
(output-stationary).
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)


def sistolik(A, B, R, C, kadr_t=None):
    """R x C sistolik massiv, output-stationary. A: (R, K), B: (K, C). Qaytaradi: natija,
taktlar, foydali MAC, kadr."""
    K = A.shape[1]
    a_reg = np.zeros((R, C)); b_reg = np.zeros((R, C)); acc = np.zeros((R, C))
    faol_mac = 0; kadr = None
    T = K + R + C - 2                      # oxirgi ko'paytma PE(R-1, C-1) ga t = (K-1)+(R-1)+
(C-1) da keladi
    for t in range(T):
        # registrlar siljiydi: a o'ngga, b pastga (bu — "sistolik" yurak urishi)
        a_reg[:, 1:] = a_reg[:, :-1].copy(); b_reg[1:, :] = b_reg[:-1, :].copy()
        for i in range(R):                 # i-satr t-i taktda boshlanadi (skew)
            k = t - i
            a_reg[i, 0] = A[i, k] if 0 <= k < K else 0.0
        for j in range(C):
            k = t - j
            b_reg[0, j] = B[k, j] if 0 <= k < K else 0.0
        faol = (a_reg != 0) & (b_reg != 0)
        faol_mac += int(faol.sum())
        acc += a_reg * b_reg
        if kadr_t == t:
            kadr = faol.copy()
    return acc, T, faol_mac, kadr
```

```
print("=== Sistolik massiv simulyatori: to'g'rilik tekshiruvi ===")
print(f"{'massiv':>7s} {'K':>4s} {'taktlar':>8s} {'formula K+R+C-2':>16s} {'MAC':>7s} {'PE-
foydalanish':>15s} {'xato':>9s}")
for R, C, K in [(4, 4, 4), (8, 8, 8), (8, 8, 64), (16, 16, 16), (16, 16, 256)]:
    A = rng.integers(1, 9, (R, K)).astype(float); B = rng.integers(1, 9, (K, C)).astype(float)
# nol yo'q — faol PE sanash aniq
    D, T, mac, _ = sistolik(A, B, R, C)
    xato = np.abs(D - A @ B).max()
    print(f"{R:>3d}x{C:<3d} {K:4d} {T:8d} {K + R + C - 2:16d} {mac:7d} {100 * mac / (R * C *
T):14.1f}% {xato:9.1e}")
print("K katta bo'lganda to'ldirish/bo'shatish (fill/drain) ulushi kamayadi va foydalanish 100
% ga yaqinlashadi.")

# --- To'lqin fronti: 8x8 massivda 6-taktda qaysi PE'lar ishlayapti ---
A = rng.integers(1, 9, (8, 8)).astype(float); B = rng.integers(1, 9, (8, 8)).astype(float)
_, _, _, kadr = sistolik(A, B, 8, 8, kadr_t=6)
print("\n=== 8x8 massiv, t=6: faol PE'lar (#) — diagonal to'lqin ===")
for i in range(8):
    print("   " + " ".join("#" if kadr[i, j] else "." for j in range(8)))

# --- TPU v4 MXU modeli: 128x128, weight-stationary; 4 MXU x 2 yadro, 1,05 GHz ---
RM = CM = 128
F = 1.05e9; MXU = 8
print(f"\nNazariy cho'qqi: {MXU} MXU x {RM}x{CM} x 2 op x {F / 1e9:.2f} GHz = {MXU * RM * CM *
2 * F / 1e12:.0f} TFLOP/s (BF16)")


def mxu_foydalanish(M, K, N, ikki_bufer=True):
    """Og'irlik plitkasi (128x128) yuklanadi, M ta faollashtirish satri oqib o'tadi.
    Har plitka: yuklash 128 takt (ikki buferda yashiriladi) + M + 2*128 - 2 takt. Bo'sh
kataklar (padding) — isrof."""
    plitkalar = math.ceil(K / RM) * math.ceil(N / CM)
    takt_plitka = M + RM + CM - 2 + (0 if ikki_bufer else RM)
    jami_takt = plitkalar * takt_plitka
    return (M * K * N) / (RM * CM * jami_takt)


QATLAM = [  # nom, M (token/piksel x batch), K, N
    ("BERT FFN, 512 token",            512, 768, 3072),
    ("BERT FFN, 4096 token (batch 8)", 4096, 768, 3072),
    ("BERT FFN, 32 token",             32, 768, 3072),
    ("ResNet conv1 (K=7*7*3=147)",     112 * 112, 147, 64),
    ("ResNet conv3x3 (K=576, N=64)",   56 * 56, 576, 64),
    ("ResNet conv3x3 (K=4608, N=512)", 7 * 7 * 32, 4608, 512),
    ("LLM dekod GEMV (M=1)",           1, 4096, 4096),
    ("LLM dekod, batch 64",            64, 4096, 4096),
    ("Toq o'lcham (N=130)",            2048, 1024, 130),
]
satr = []
for nom, M, K, N in QATLAM:
    u = mxu_foydalanish(M, K, N)
    satr.append({"qatlam": nom, "M": M, "K": K, "N": N, "foydalanish_%": 100 * u,
                 "samarali_TFLOP/s": 275 * u, "ikki_bufersiz_%": 100 * mxu_foydalanish(M, K, N,
False)})
df = pd.DataFrame(satr)
print("\n=== TPU v4 MXU foydalanish modeli (analitik, taxminiy) ===")
print(df.to_string(index=False, float_format=lambda v: f"{v:.1f}"))
print("Xulosa: M kichik (dekod, kichik batch) yoki K/N 128 ga karrali bo'lmasa, MXU bo'sh
turadi;")
print("XLA kompilyatori shuning uchun batch va o'lchamlarni 128 ga moslashni (padding) talab
qiladi.")
```

```
# --- Grafik: foydalanish vs M (batch x token) ---
Ms = [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192]
fig, ax = plt.subplots(figsize=(8, 4.5))
for K, N in [(768, 3072), (4096, 4096), (147, 64)]:
    ax.semilogx(Ms, [100 * mxu_foydalanish(m, K, N) for m in Ms], marker="o", label=f"K={K}, N=
{N}")
ax.set_xlabel("M (batch x token / piksel)"); ax.set_ylabel("MXU foydalanish, %")
ax.set_title("Sistolik massiv 128x128: foydalanish M ga bog'liq"); ax.grid(alpha=0.3);
ax.legend()
plt.tight_layout()
print("\n=== Simulyatsiya tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Sistolik massiv simulyatori`, `=== TPU v4 MXU foydalanish modeli`, `=== Simulyatsiya tugadi ===`

TAJRIBA QILIB KO‘RING

- Massivni weight-stationary rejimga o'zgartiring (B oldindan PE'larga yuklanadi) va taktlar formulasini chiqaring.
- MXU o'lchamini 256x256 ga oshiring (TPU v6e kabi) — kichik qatlamlarda foydalanish qanday o'zgaradi?
- Toq o'lcham N=130 uchun 128 ga moslash (padding) va 2 ta plitka o'rniga N ni 128 ga qisqartirish samarasini solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. FPGA: quvur (pipeline), II va fixed-point aniqlik simulyatsiyasi

MAQSAD

Birinchi qismda 4 bosqichli MAC quvuri vaqt diagrammasi bilan simulyatsiya qilinadi: initiation interval (II) 1, 2, 4 bo'lganda o'tkazuvchanlik qanday tushishi ko'rsatiladi, Alveo U250 (~12288 DSP, 300 MHz) uchun INT16/INT8/FP32 rejimlarda TOP/s bahosi chiqariladi. Ikkinchi qismda digits ustida o'qitilgan MLP 64-32-10 butunlay butun sonli (fixed-point) arifmetikada bajariladi: Qm.n formatlari, to'yinish (saturation), siljitish orqali masshtab va akkumulyator kengligi. Har format uchun aniqlik, SQNR va to'yingan og'irliklar soni jadvalda.

> **Kutilgan natija:** II=1 da 8 kirish 11 taktda; INT8 packing ~12 TOP/s (taxminiy); 16 bitda aniqlik suzuvchi nuqta bilan bir xil (~0,97), Q0.7 da to'yinish va keskin tushish, 12 bitli akkumulyatorda ~0,6.

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. FPGA: quvur (pipeline) va fixed-point aniqlik simulyatsiyasi
# (a) Takt darajasidagi quvur: MAC yo'li 4 bosqichga bo'lingan (o'qish -> ko'paytirish ->
qo'shish -> yozish).
#     II (initiation interval) = yangi kirish qabul qilish oralig'i. Throughput = f_clk *
parallel / II.
# (b) Fixed-point: digits MLP og'irlik va faollashtirishlarini Qm.n formatga keltirib,
to'yinishli (saturating)
#     butun sonli arifmetikada bajaramiz — FPGA'dagi DSP bloklari aynan shunday hisoblaydi.
import warnings; warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

# ---------------- (a) Quvur simulyatori ----------------
def quvur(n_kirish, bosqich=4, II=1):
    """Har kirish II taktda bir qabul qilinadi; har bosqich 1 takt. Qaytaradi: jami taktlar,
vaqt chizig'i."""
    boshlash = [i * II for i in range(n_kirish)]
    tugash = [b + bosqich for b in boshlash]
    return tugash[-1], boshlash


print("=== (a) Quvur: 4 bosqichli MAC, 8 ta kirish ===")
for II in (1, 2, 4):
    T, bosh = quvur(8, 4, II)
    print(f"II={II}: jami {T:3d} takt; o'tkazuvchanlik {8 / T:.2f} natija/takt")
T, bosh = quvur(6, 4, 1)
print("\nVaqt diagrammasi (II=1): O-o'qish, K-ko'paytirish, Q-qo'shish, Y-yozish")
for i, b in enumerate(bosh):
    print(f"kirish {i}: " + "." * b + "OKQY" + "." * (T - b - 4))
print("Quvursiz (har kirish 4 takt, ketma-ket): 6 kirish = 24 takt; quvur bilan = "
      f"{T} takt -> {24 / T:.1f}x tezroq")

# --- FPGA resurs/o'tkazuvchanlik modeli: Alveo U250 (~12288 DSP, taxminiy), 300 MHz ---
print("\n=== FPGA o'tkazuvchanlik modeli (taxminiy) ===")
```

```
DSP, F = 12288, 300e6
print(f"{'rejim':34s} {'MAC/DSP':>8s} {'band DSP':>9s} {'II':>3s} {'TOP/s':>7s}")
for nom, mac_dsp, ulush, II in [("INT16, 1 MAC/DSP", 1, 0.8, 1), ("INT8, 2 MAC/DSP (packing)",
2, 0.8, 1),
                                ("INT8, II=2 (xotira cheklovi)", 2, 0.8, 2), ("FP32 (~3
DSP/MAC)", 1 / 3, 0.8, 1)]:
    tops = DSP * ulush * mac_dsp * 2 * F / II / 1e12
    print(f"{nom:34s} {mac_dsp:8.2f} {int(DSP * ulush):9d} {II:3d} {tops:7.2f}")
print("Ishlab chiqaruvchi INT8 cho'qqisi (33,3 TOP/s) yuqoriroq chastota va to'liq DSP
foydalanishni nazarda tutadi.")

# ---------------- (b) Fixed-point aniqlik ----------------
X, y = load_digits(return_X_y=True); X = (X / 16.0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
mlp = MLPClassifier(hidden_layer_sizes=(32,), max_iter=300, random_state=0).fit(Xtr, ytr)
W1, W2 = mlp.coefs_; b1, b2 = mlp.intercepts_
acc_fp = (mlp.predict(Xte) == yte).mean()


def qkvant(x, butun, kasr, jami):
    """Ishorali fixed-point: jami bit, 'kasr' bit kasr qism. To'yinish (saturation) bilan butun
songa."""
    q = np.round(x * (1 << kasr))
    lo, hi = -(1 << (jami - 1)), (1 << (jami - 1)) - 1
    return np.clip(q, lo, hi).astype(np.int64), int(((q < lo) | (q > hi)).sum())


def fixed_mlp(X, jami, kasr_w, kasr_a, akk_bit):
    """Butun sonli inferens: x(Q kasr_a) * w(Q kasr_w) -> akkumulyator (Q kasr_a+kasr_w),
to'yinish akk_bit da."""
    xq, _ = qkvant(X, 0, kasr_a, jami)
    w1q, s1 = qkvant(W1, 0, kasr_w, jami); w2q, s2 = qkvant(W2, 0, kasr_w, jami)
    lo, hi = -(1 << (akk_bit - 1)), (1 << (akk_bit - 1)) - 1
    h = np.clip(xq @ w1q + np.round(b1 * (1 << (kasr_a + kasr_w))).astype(np.int64), lo, hi)
    h = np.maximum(h, 0) >> kasr_w                      # ReLU va masshtabni qaytarish
(siljitish — FPGA'da bepul)
    h = np.clip(h, -(1 << (jami - 1)), (1 << (jami - 1)) - 1)
    o = np.clip(h @ w2q + np.round(b2 * (1 << (kasr_a + kasr_w))).astype(np.int64), lo, hi)
    hf = np.maximum(X @ W1 + b1, 0); of = hf @ W2 + b2   # suzuvchi nuqta etalon
    oq = o / float(1 << (kasr_a + kasr_w))
    sqnr = 10 * np.log10((of ** 2).sum() / max(((of - oq) ** 2).sum(), 1e-30))
    return (o.argmax(1) == yte).mean(), sqnr, s1 + s2


print(f"\n=== (b) Fixed-point MLP 64-32-10 (digits), suzuvchi nuqta aniqligi: {acc_fp:.4f}
===")
print(f"Og'irliklar diapazoni: [{min(W1.min(), W2.min()):.2f}, {max(W1.max(), W2.max()):.2f}]")
satr = []
for jami, kw, ka, akk in [(16, 12, 12, 48), (12, 8, 8, 32), (8, 6, 6, 24), (8, 4, 5, 24), (8,
7, 7, 24),
                          (6, 4, 4, 20), (4, 2, 2, 16), (8, 6, 6, 12)]:
    a, sq, sat = fixed_mlp(Xte, jami, kw, ka, akk)
    satr.append({"bit": jami, "Q_w": f"Q{jami - kw - 1}.{kw}", "Q_a": f"Q{jami - ka - 1}.{ka}",
"akk_bit": akk,
                 "aniqlik": a, "farq_pp": 100 * (a - acc_fp), "SQNR_dB": sq, "to'yingan_w":
sat})
print(pd.DataFrame(satr).to_string(index=False, float_format=lambda v: f"{v:.3f}"))
print("Kuzating: kasr bit ko'paysa butun qism qisqaradi -> katta og'irliklar to'yinadi
(Q0.7);")
print("akkumulyator torligi (12 bit) natijani buzadi — FPGA'da akkumulyator kengligini to'g'ri
tanlash shart.")
print("\n=== FPGA simulyatsiyasi tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== (a) Quvur`, `=== FPGA o'tkazuvchanlik modeli`, `=== (b) Fixed-point MLP`, `=== FPGA simulyatsiyasi tugadi ===`

TAJRIBA QILIB KO‘RING

- Quvurga 2 taktli xotira o'qish bosqichini qo'shing (5 bosqich) — kechikish va o'tkazuvchanlik qanday o'zgaradi?
- Yashirin qatlamni 128 neyronga oshiring: 8 bitli akkumulyator yetarli bo'lishi uchun nechta bit kerak? (log2(K) qoidasi)
- Har qatlamga alohida Q format tanlang (qatlam bo'yicha diapazon) va aniqlikni solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. NPU uchun INT8 kvantlash: aniqlik, hajm, kechikish va grafik bo'linishi

MAQSAD

Kichik CNN digits ustida o'qitilib, PyTorch eager statik kvantlash (qnnpack) bilan INT8 ga o'tkaziladi: conv+relu birlashtiriladi, per-tensor va per-channel og'irlik kvantlash, 8 va 256 namunali kalibrlash solishtiriladi. Aniqlik, model hajmi va batch 1/64 kechikishi o'lchanadi. So'ng har qatlam og'irliklarining simmetrik INT8 SQNR'i hisoblanadi. Oxirida NPU qo'llamaydigan operatorlar (SiLU, NMS) CPU'ga qaytganda PCIe orqali qatnov kechikishi modellashtiriladi va SiLU -> ReLU almashtirish samarasi ko'rsatiladi.

> **Kutilgan natija:** FP32 va INT8 aniqligi deyarli teng (~0,95), hajm ~3,5x kichik; CPU'da batch=1 da INT8 tez bo'lmasligi mumkin, batch 64 da tezroq; SQNR ~40-47 dB; bo'linish modelida o'tishlar 5 -> 3, kechikish ~3,0 -> 2,4 ms.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. NPU uchun INT8 kvantlash: aniqlik, hajm, kechikish va operatorlarni bo'lish
(partitioning)
# NPU'lar (Hailo, Edge TPU, Ethos, DLA) asosan INT8 da ishlaydi. Biz kichik CNN'ni PyTorch
statik kvantlash
# (qnnpack) bilan INT8 ga o'tkazamiz: per-tensor vs per-channel, kalibrlash hajmi ta'siri va
NPU qo'llab-quvvatlamaydigan
# operatorlar CPU'ga qaytganda (fallback) qancha vaqt yo'qotilishini modellashtiramiz.
import warnings; warnings.filterwarnings("ignore")
import copy, io, time
import numpy as np
import torch, torch.nn as nn
import torch.ao.quantization as tq
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); torch.set_num_threads(2)
torch.backends.quantized.engine = "qnnpack"      # AVX'siz CPU uchun
torch.backends.nnpack.set_flags(False)

X, y = load_digits(return_X_y=True); X = (X / 16.0).astype(np.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
Ttr, Tytr, Tte = torch.tensor(Xtr), torch.tensor(ytr), torch.tensor(Xte)


class CNN(nn.Module):
    def __init__(s):
        super().__init__()
        s.quant = tq.QuantStub(); s.dequant = tq.DeQuantStub()
        s.c1 = nn.Conv2d(1, 16, 3, padding=1); s.r1 = nn.ReLU()
        s.c2 = nn.Conv2d(16, 32, 3, padding=1); s.r2 = nn.ReLU(); s.p = nn.MaxPool2d(2)
        s.fc1 = nn.Linear(32 * 4 * 4, 64); s.r3 = nn.ReLU(); s.fc2 = nn.Linear(64, 10)

    def forward(s, x):
        x = s.quant(x)
        x = s.p(s.r2(s.c2(s.r1(s.c1(x)))))
        x = s.r3(s.fc1(x.reshape(x.shape[0], -1)))
        return s.dequant(s.fc2(x))
```

```
m = CNN(); opt = torch.optim.Adam(m.parameters(), 2e-3); lf = nn.CrossEntropyLoss()
for ep in range(6):
    perm = torch.randperm(len(Ttr))
    for i in range(0, len(Ttr), 64):
        idx = perm[i:i + 64]; opt.zero_grad(); lf(m(Ttr[idx]), Tytr[idx]).backward();
opt.step()
m.eval()


def aniqlik(model):
    with torch.no_grad():
        return (model(Tte).argmax(1).numpy() == yte).mean()


def hajm_kb(model):
    b = io.BytesIO(); torch.save(model.state_dict(), b); return len(b.getvalue()) / 1024


def kechikish_ms(model, batch=1, n=300):
    x = Tte[:batch]
    with torch.no_grad():
        for _ in range(20): model(x)
        t = []
        for _ in range(n):
            t0 = time.perf_counter(); model(x); t.append(time.perf_counter() - t0)
    return 1e3 * float(np.median(t))


def kvantla(model, qconfig, n_kalib):
    q = copy.deepcopy(model); q.qconfig = qconfig
    q = tq.fuse_modules(q, [["c1", "r1"], ["c2", "r2"], ["fc1", "r3"]])
    tq.prepare(q, inplace=True)
    with torch.no_grad():
        q(Ttr[:n_kalib])                      # kalibrlash: faollashtirish diapazonlari
yig'iladi
    tq.convert(q, inplace=True)
    return q


per_tensor = tq.QConfig(activation=tq.MinMaxObserver.with_args(dtype=torch.quint8),
                        weight=tq.MinMaxObserver.with_args(dtype=torch.qint8,
qscheme=torch.per_tensor_symmetric))
per_channel = tq.QConfig(activation=tq.MinMaxObserver.with_args(dtype=torch.quint8),
                         weight=tq.PerChannelMinMaxObserver.with_args(dtype=torch.qint8,
qscheme=torch.per_channel_symmetric))

print("=== INT8 statik kvantlash (qnnpack, CPU — NPU o'rniga) ===")
print(f"{'model':32s} {'aniqlik':>8s} {'hajm KB':>8s} {'b=1 ms':>8s} {'b=64 ms':>8s}")
print(f"{'FP32':32s} {aniqlik(m):8.4f} {hajm_kb(m):8.1f} {kechikish_ms(m):8.3f}
{kechikish_ms(m, 64, 100):8.3f}")
for nom, qc in (("INT8 per-tensor", per_tensor), ("INT8 per-channel", per_channel)):
    for nk in (8, 256):
        q = kvantla(m, qc, nk)
        print(f"{nom + f', kalib={nk}':32s} {aniqlik(q):8.4f} {hajm_kb(q):8.1f}
{kechikish_ms(q):8.3f} {kechikish_ms(q, 64, 100):8.3f}")
print("Hajm ~4x kamayadi. CPU'da INT8 har doim tezroq emas (qnnpack, kichik model) — NPU'da esa
INT8 yagona tez yo'l.")

# --- Kvantlash xatosi qatlam bo'yicha (simmetrik per-tensor, NPU uslubi) ---
print("\n=== Qatlam og'irliklari: INT8 kvantlash xatosi (SQNR, dB) ===")
for nom, p in m.named_parameters():
```

```
    if p.dim() < 2: continue
    w = p.detach().numpy(); s = np.abs(w).max() / 127
    wq = np.clip(np.round(w / s), -127, 127) * s
    sqnr = 10 * np.log10((w ** 2).sum() / ((w - wq) ** 2).sum())
    print(f"{nom:12s} shakl={str(tuple(w.shape)):18s} masshtab={s:.5f}  SQNR={sqnr:5.1f} dB")

# --- Operatorlarni bo'lish: NPU qo'llamaydigan op CPU'da bajariladi, tenzor PCIe orqali
qatnaydi ---
print("\n=== NPU grafik bo'linishi modeli (taxminiy, Hailo-8 kabi M.2 NPU) ===")
NPU_TOPS, PCIE_GBS, CPU_GOPS = 26 * 0.3, 3.0, 50   # NPU real foydalanish ~30 %, PCIe ~3 GB/s,
xost CPU ~50 GOP/s
OPLAR = [  # (nom, amallar GOP, chiqish tenzori MB, NPU qo'llaydimi)
    ("conv_stem", 0.9, 1.6, True), ("conv_blok1", 2.1, 0.8, True), ("SiLU (qo'llanmaydi)",
0.01, 0.8, False),
    ("conv_blok2", 2.4, 0.4, True), ("NMS/postprocess", 0.05, 0.05, False), ("conv_head", 0.6,
0.2, True)]


def ms(gop, npu):  # amal vaqti, ms
    return gop / (NPU_TOPS * 1e3) * 1e3 if npu else gop / CPU_GOPS * 1e3


RELU = [("ReLU", g, mb, True) if "SiLU" in n else (n, g, mb, npu) for n, g, mb, npu in OPLAR]
# SiLU -> ReLU (NPU qo'llaydi)
for nom_v, oplar in (("asl grafik (SiLU + NMS CPU'da)", OPLAR), ("SiLU -> ReLU almashtirilgan",
RELU)):
    jami = 0.0; joy = "CPU"; utish = 0           # kirish tasviri xost (CPU) xotirasida
    for nom, gop, mb, npu in oplar:
        yangi = "NPU" if npu else "CPU"
        if yangi != joy:
            jami += mb / PCIE_GBS; utish += 1
        jami += ms(gop, npu); joy = yangi
    print(f"{nom_v:34s} kechikish ~{jami:6.2f} ms, CPU<->NPU o'tishlar: {utish}")
print("Xulosa: bitta qo'llanmaydigan aktivatsiya grafikni bo'lib, PCIe orqali qo'shimcha
nusxalashga olib keladi.")
print("\n=== Kvantlash tahlili tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== INT8 statik kvantlash`, `=== Qatlam og'irliklari`, `=== NPU grafik bo'linishi modeli`, `=== Kvantlash tahlili tugadi ===`

TAJRIBA QILIB KO‘RING

- Kalibrlash uchun HistogramObserver ishlating va aniqlik farqini o'lchang.
- Modelga SiLU qo'shing va kvantlangan model qaysi qatlamda dequant/quant qilishini ko'ring (print(q)).
- PCIE_GBS ni 0,5 GB/s (USB) ga o'zgartiring — Edge TPU USB kabi qurilmada bo'linish narxi qancha?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Real CPU'da GEMM va konvolyutsiya o'lchovlari, boshqa arxitekturalarga ekstrapolyatsiya

MAQSAD

Sandbox CPU (2 yadro) uchun haqiqiy Roofline quriladi: xotira o'tkazuvchanligi STREAM triad bilan, amaliy cho'qqi esa eng yaxshi FP32 GEMM bilan o'lchanadi. GEMM 64...1024, GEMV, conv3x3, conv1x1, depthwise va ReLU uchun GFLOP/s, arifmetik intensivlik va Roofline'ga nisbatan samaradorlik jadvali chiqadi. Keyin har yadro samaradorligi saqlanadi deb faraz qilinib, 1-qadam pasportidagi arxitekturalarda kutilgan vaqt (ishga tushirish xarajati bilan) ekstrapolyatsiya qilinadi. Bu — birinchi yaqinlashish, aniq bashorat emas.

> **Kutilgan natija:** Triad ~20-40 GB/s, GEMM cho'qqisi ~40-60 GFLOP/s (mashinaga bog'liq); depthwise samaradorligi < 10 %; ekstrapolyatsiyada kichik yadrolar tezlatkichlarda 5-10 us 'pol'ga uriladi.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. Real CPU'da GEMM va konvolyutsiya o'lchovlari, o'lchangan Roofline va boshqa
arxitekturalarga ekstrapolyatsiya
# Sandbox CPU (2 yadro) — bizning "laboratoriya qurilmamiz". Avval uning haqiqiy cho'qqisi (eng
yaxshi GEMM) va
# xotira o'tkazuvchanligi (STREAM triad) o'lchanadi, keyin har yadroning samaradorligi
(o'lchangan / Roofline chegarasi)
# topiladi va shu samaradorlik bilan GPU/TPU/NPU/FPGA'dagi vaqt TAXMINIY bashorat qilinadi.
import time
import numpy as np
import pandas as pd
import torch
import torch.nn.functional as Fn
import matplotlib.pyplot as plt

torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)


def olch(f, takror=7):
    f(); f()
    t = []
    for _ in range(takror):
        t0 = time.perf_counter(); f(); t.append(time.perf_counter() - t0)
    return float(np.median(t))


# --- 1) Xotira o'tkazuvchanligi: STREAM triad a = b + s*c (float32, 3 massiv x 16 MB) ---
n = 4 * 1024 * 1024
b = torch.rand(n); c = torch.rand(n); a = torch.empty(n)
t = olch(lambda: torch.add(b, c, alpha=1.5, out=a), 9)
BW = 3 * n * 4 / t / 1e9
print("=== Sandbox CPU o'lchovlari ===")
print(f"STREAM triad o'tkazuvchanligi: {BW:6.2f} GB/s")

# --- 2) GEMM FP32: o'lcham bo'yicha GFLOP/s ---
natija = []
for N in (64, 128, 256, 512, 1024):
    A = torch.randn(N, N); B = torch.randn(N, N)
```

```
    t = olch(lambda: A @ B, 5 if N < 1024 else 3)
    ops, bayt = 2 * N ** 3, 4 * 3 * N * N
    natija.append(("GEMM %d" % N, ops, bayt, t))
# GEMV (LLM dekod kabi) — xotira chegarasida
W = torch.randn(4096, 2048); v = torch.randn(2048)
t = olch(lambda: W @ v, 9); natija.append(("GEMV 4096x2048", 2 * 4096 * 2048, 4 * (4096 * 2048
+ 2048 + 4096), t))

# --- 3) Konvolyutsiyalar (batch 8) ---
for nom, Ci, Co, H, k, g in (("conv3x3 32->64 28x28", 32, 64, 28, 3, 1), ("conv1x1 64->128
28x28", 64, 128, 28, 1, 1),
                             ("depthwise 3x3 64ch 56x56", 64, 64, 56, 3, 64)):
    x = torch.randn(8, Ci, H, H); w = torch.randn(Co, Ci // g, k, k)
    t = olch(lambda: Fn.conv2d(x, w, padding=k // 2, groups=g), 5)
    ops = 2 * 8 * H * H * Co * (Ci // g) * k * k
    bayt = 4 * (x.numel() + w.numel() + 8 * Co * H * H)
    natija.append((nom, ops, bayt, t))
# --- element bo'yicha: ReLU ---
z = torch.randn(8, 64, 56, 56)
t = olch(lambda: torch.relu(z), 9); natija.append(("ReLU 8x64x56x56", z.numel(), 8 * z.numel(),
t))

df = pd.DataFrame(natija, columns=["yadro", "ops", "bayt", "t_s"])
df["GFLOP/s"] = df.ops / df.t_s / 1e9
df["AI"] = df.ops / df.bayt
PEAK = df["GFLOP/s"].max()                                  # o'lchangan "amaliy cho'qqi"
df["roofline"] = np.minimum(PEAK, df.AI * BW)
df["samaradorlik_%"] = 100 * df["GFLOP/s"] / df["roofline"]   # >100 % — ma'lumot keshga
sig'gan (triad BW dan tez)
print(f"Amaliy cho'qqi (eng yaxshi GEMM): {PEAK:6.1f} GFLOP/s, ridge nuqtasi = {PEAK / BW:5.1f}
FLOP/bayt\n")
print(df[["yadro", "AI", "GFLOP/s", "roofline", "samaradorlik_%"]].to_string(index=False,
float_format=lambda v: f"{v:.2f}"))

# --- 4) Ekstrapolyatsiya: bir xil yadro boshqa arxitekturada (taxminiy cho'qqilar, FP16/INT8
rejimida) ---
ARX = {  # nom: (cho'qqi TOP/s, BW GB/s, ishga tushirish xarajati us — TAXMINIY faraz)  — 1-
qadam pasportidan
    "EPYC 9654 (INT8)": (29.5, 460.8, 0), "H100 (FP16)": (989.0, 3350.0, 5), "TPU v4 (BF16)":
(275.0, 1200.0, 5),
    "Jetson AGX Orin (INT8)": (138.0, 204.8, 8), "Alveo U250 (INT8)": (33.3, 77.0, 10), "Hailo-
8 (INT8)": (26.0, 4.0, 50)}
print("100 % dan katta samaradorlik: matritsa L2/L3 keshga qisman sig'gan — RAM
o'tkazuvchanligidan tezroq o'qilgan.")
print("\n=== Ekstrapolyatsiya: kutilgan vaqt, mikrosoniya (samaradorlik sandbox'dagidek + ishga
tushirish xarajati) ===")
satr = []
for _, r in df.iterrows():
    s = {"yadro": r.yadro, "sandbox_us": 1e6 * r.t_s}
    e = min(max(r["samaradorlik_%"], 5), 100) / 100           # samaradorlik 5..100 %
oralig'ida
    for nom, (pk, bw, ust) in ARX.items():
        bayt_k = r.bayt / 4 * (2 if "FP16" in nom or "BF16" in nom else 1)   # FP32 -> 2 yoki 1
bayt/element
        s[nom.split(" (")[0]] = ust + 1e6 * r.ops / (e * min(pk * 1e12, r.ops / bayt_k * bw *
1e9))
    satr.append(s)
print(pd.DataFrame(satr).to_string(index=False, float_format=lambda v: f"{v:.2f}"))
print("Ogohlantirish: bu birinchi yaqinlashish. Kichik yadrolarda tezlatkich vaqtini ishga
tushirish xarajati ('pol') belgilaydi —")
print("shuning uchun tezlatkichlarda yadrolarni birlashtirish (fusion) va katta batch muhim.
Xostdan nusxa (PCIe) hisobga olinmagan.")
```

```
# --- Grafik: o'lchangan Roofline ---
fig, ax = plt.subplots(figsize=(8, 4.8))
ai = np.logspace(-1.5, 3, 100)
ax.loglog(ai, np.minimum(PEAK, ai * BW), "k-", label=f"sandbox Roofline ({PEAK:.0f} GFLOP/s,
{BW:.1f} GB/s)")
for _, r in df.iterrows():
    ax.loglog(r.AI, r["GFLOP/s"], "o"); ax.annotate(r.yadro, (r.AI, r["GFLOP/s"]), fontsize=7)
ax.set_xlabel("AI, FLOP/bayt"); ax.set_ylabel("GFLOP/s"); ax.set_title("Sandbox CPU: o'lchangan
Roofline")
ax.grid(True, which="both", alpha=0.3); ax.legend(fontsize=8); plt.tight_layout()
print("\n=== O'lchovlar tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Sandbox CPU o'lchovlari ===`, `Amaliy cho'qqi`, `=== Ekstrapolyatsiya`, `=== O'lchovlar tugadi ===`

TAJRIBA QILIB KO‘RING

- torch.set_num_threads(1) bilan qayta o'lchang — cho'qqi va ridge nuqtasi qanday o'zgaradi?
- GEMM'ni float64 da o'lchang (AI 2 marta kamayadi) va Roofline'dagi joyini ko'ring.
- Ishga tushirish xarajatini 0 qiling — qaysi yadrolar uchun bashorat keskin o'zgaradi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Energiya samaradorligi (TOP/s/Vt, mJ/tasvir) va 3 yillik TCO

MAQSAD

ResNet-50 sinfidagi tasnif (~8,2 GOP/tasvir) va 5000 tasvir/s talab uchun har arxitekturada kerakli qurilmalar soni, umumiy quvvat, energiya (mJ/tasvir), CAPEX, elektr xarajati (PUE bilan) va egalik/bulut TCO hisoblanadi; natija bitta mln tasvir narxiga keltiriladi. Elektr narxi bo'yicha sezgirlik, 30 % yuklamada bulut va egalik solishtiruvi hamda chekka stsenariysi (120 tasvir/s, 30 Vt) ko'rib chiqiladi. Narx, quvvat va real foydalanish koeffitsientlari — taxminiy faraz, ular kod boshida o'zgartiriladi.

> **Kutilgan natija:** TOP/s/Vt: NPU ~10, GPU ~2,8, CPU ~0,08; mJ/tasvir NPU ~3,5, GPU ~10, CPU ~470; 24/7 da GPU egaligi bulutdan arzon, 30 % yuklamada bulut afzal; chekkada faqat NPU mos.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Energiya samaradorligi (TOP/s/Vt, J/inferens) va 3 yillik TCO taqqoslash
# Ssenariy: ResNet-50 sinfidagi tasvir tasnifi (~4,1 GMAC = 8,2 GOP/tasvir), talab — 5000
tasvir/s, 24/7.
# Barcha narx, quvvat va "real foydalanish" koeffitsientlari TAXMINIY faraz — o'z
qiymatlaringizni qo'ying.
import math
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option("display.width", 170)
GOP_TASVIR = 8.2           # ResNet-50, bitta tasvir, GOP
TALAB = 5000               # tasvir/s
SOAT = 3 * 365 * 24        # 3 yil, soat
NARX_KVT = 0.10            # USD / kVt*soat (taxminiy)
PUE = 1.4                  # ma'lumot markazi: sovutish va boshqa yo'qotishlar koeffitsienti

# nom: cho'qqi INT8 TOP/s, real foydalanish (MLPerf'dagi kabi tartib, taxminiy), yuklamadagi
quvvat Vt,
#      karta narxi USD, xost ulushi USD, xost quvvat ulushi Vt, bulut USD/soat (yo'q = None)
Q = {
    "CPU EPYC 9654":    (29.5, 0.30, 360, 11800, 4000, 150, None),
    "GPU H100 SXM":     (1979.0, 0.35, 700, 30000, 6000, 150, 3.00),
    "TPU v4 (bulut)":   (275.0, 0.50, 200, None, None, 0, 3.22),
    "NPU Hailo-8":      (26.0, 0.50, 2.5, 250, 300, 3, None),
    "Jetson AGX Orin":  (138.0, 0.30, 60, 2000, 0, 0, None),
    "FPGA Alveo U250":  (33.3, 0.60, 225, 9000, 3000, 100, None),
}
satr = []
for nom, (pk, u, vt, narx, xost, xvt, bulut) in Q.items():
    ips = pk * u * 1e3 / GOP_TASVIR                    # tasvir/s bitta qurilmada
    n = math.ceil(TALAB / ips)
    P = n * (vt + xvt)                                 # umumiy quvvat, Vt
    energiya_kvt = P * SOAT / 1e3 * PUE
    capex = None if narx is None else n * (narx + xost)
    opex_el = energiya_kvt * NARX_KVT
    tco_own = None if capex is None else capex + opex_el
    tco_cloud = None if bulut is None else n * bulut * SOAT
    tco = min(x for x in (tco_own, tco_cloud) if x is not None)
    satr.append({"qurilma": nom, "TOPS/Vt (cho'qqi)": pk / vt, "tasvir/s/qurilma": ips, "soni":
n,
```

```
                "quvvat_kVt": P / 1e3, "mJ/tasvir": 1e3 * (vt + xvt) / ips, "CAPEX_k$": (capex
or 0) / 1e3,
                "elektr_k$": opex_el / 1e3, "egalik_TCO_k$": None if tco_own is None else
tco_own / 1e3,
                "bulut_TCO_k$": None if tco_cloud is None else tco_cloud / 1e3,
                "c/mln tasvir": 100 * tco / (TALAB * SOAT * 3600 / 1e6)})     # sent (cent) /
1 mln tasvir
df = pd.DataFrame(satr)
print(f"=== Energiya va TCO: talab {TALAB} tasvir/s, 3 yil, {NARX_KVT} $/kVt*soat, PUE {PUE}
(taxminiy) ===")
print(df.to_string(index=False, float_format=lambda v: f"{v:.2f}", na_rep="—"))

# --- Sezgirlik: elektr narxi va o'rtacha yuklama (load factor) TCO ga qanday ta'sir qiladi ---
print("\n=== Sezgirlik: egalik TCO (k$) elektr narxiga qarab ===")
print(f"{'qurilma':18s}" + "".join(f"{p:>10.2f}$" for p in (0.05, 0.10, 0.20, 0.40)))
for _, r in df.iterrows():
    if pd.isna(r["egalik_TCO_k$"]): continue
    capex = r["CAPEX_k$"]; el = r["elektr_k$"]
    print(f"{r.qurilma:18s}" + "".join(f"{capex + el * p / NARX_KVT:11.1f}" for p in (0.05,
0.10, 0.20, 0.40)))

print("\n=== Yuklama 30 % bo'lsa (kunduzgi cho'qqi, tunda bo'sh): bulut vs egalik ===")
for _, r in df.iterrows():
    if pd.isna(r["bulut_TCO_k$"]): continue
    own = "—" if pd.isna(r["egalik_TCO_k$"]) else f"{r['egalik_TCO_k$']:.1f}"
    print(f"{r.qurilma:18s} egalik {own:>8s} k$ | bulut (faqat ish vaqti) "
          f"{0.3 * r['bulut_TCO_k$']:8.1f} k$")
print("Xulosa: doimiy 24/7 yuklamada egalik arzonroq bo'lishi mumkin, o'zgaruvchan yuklamada
bulut afzal.")

# --- Chekka (edge) cheklovi: 4 kamera x 30 kadr/s, quvvat byudjeti 30 Vt ---
print("\n=== Chekka stsenariysi: 120 tasvir/s, quvvat <= 30 Vt ===")
for nom, (pk, u, vt, narx, xost, xvt, bulut) in Q.items():
    ips = pk * u * 1e3 / GOP_TASVIR; n = math.ceil(120 / ips); P = n * (vt + xvt)
    holat = "MOS" if P <= 30 and bulut is None else ("bulut — chekka emas" if narx is None else
"quvvat oshadi")
    print(f"{nom:18s} {n:3d} dona, {P:7.1f} Vt -> {holat}")

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].bar(df.qurilma, df["mJ/tasvir"]); ax[0].set_yscale("log"); ax[0].set_title("Energiya,
mJ/tasvir (log)")
ax[1].bar(df.qurilma, df["c/mln tasvir"], color="tab:orange"); ax[1].set_title("Narx, sent / 1
mln tasvir")
for a in ax: a.tick_params(axis="x", rotation=45, labelsize=7)
plt.tight_layout()
print("\n=== TCO tahlili tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Energiya va TCO`, `=== Sezgirlik`, `=== Chekka stsenariysi`, `=== TCO tahlili tugadi ===`

TAJRIBA QILIB KO‘RING

- Real foydalanish koeffitsientlarini MLPerf Inference natijalari asosida yangilang (manbani hisobotda ko'rsating).
- Talabni 500 tasvir/s ga tushiring — qaysi arxitektura eng arzon bo'ladi?
- Jetson'ni 30 Vt rejimida (unumdorlik ~0,5x) hisoblang va chekka cheklovini qayta tekshiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Yakuniy tanlov matritsasi: WSM, TOPSIS va vaznlar sezgirligi

MAQSAD

Oldingi qadamlar natijalari 6 mezonli jadvalga yig'iladi (unumdorlik, kechikish, energiya, TCO, ekotizim, moslashuvchanlik) va qat'iy cheklovlar (quvvat, internetsiz ishlash) qo'llanadi. Uch ssenariy (data-markaz, chekka kamera, moliya) uchun vaznli yig'indi (WSM) va TOPSIS reytinglari hisoblanadi; Dirichlet taqsimoti bilan vaznlar 2000 marta tebratilib, 1-o'rin barqarorligi baholanadi. Natija issiqlik xaritasi ko'rinishidagi tanlov matritsasi.

> **Kutilgan natija:** Data-markaz va moliyada GPU 1-o'rin (barqarorlik 80-98 %), chekkada Jetson/NPU — usullar kelishmasligi ko'rsatiladi; matritsa va grafik.

DASTUR KODI (FAYL: S7_*.PY)

```
# 7-qadam. Yakuniy tanlov matritsasi: ko'p mezonli qaror (WSM + TOPSIS) va vaznlar sezgirligi
(Monte-Karlo)
# Oldingi qadamlar natijalari (taxminiy) mezonlar jadvaliga yig'iladi. 3 ta ssenariy uchun
turli vaznlar bilan
# arxitekturalar reytingi tuziladi, qat'iy cheklovlar (quvvat, bulutsiz ishlash) filtr sifatida
qo'llanadi.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
pd.set_option("display.width", 160)

# Mezonlar: (qiymat yo'nalishi: +1 — ko'pi yaxshi, -1 — kami yaxshi)
MEZON = {"unumdorlik": +1, "kechikish": -1, "energiya": -1, "TCO": -1, "ekotizim": +1,
"moslashuvchanlik": +1}
# Qiymatlar: unumdorlik — tasvir/s/qurilma (6-qadam), kechikish — batch=1 ms (taxminiy),
energiya — mJ/tasvir (6-qadam),
# TCO — sent / 1 mln tasvir (6-qadam), ekotizim va moslashuvchanlik — ekspert bahosi 1..5
(dasturlash qulayligi,
# kutubxonalar, yangi operatorlarni qo'llab-quvvatlash). O'z bahoingizni qo'ying!
JADVAL = pd.DataFrame({
    "unumdorlik":       [1080, 84000, 16800, 1590, 5050, 2440],
    "kechikish":        [6.0, 1.0, 1.5, 2.5, 2.0, 1.2],
    "energiya":         [473, 10.1, 11.9, 3.5, 11.9, 133],
    "TCO":              [18.7, 8.3, 17.9, 0.5, 0.5, 8.4],
    "ekotizim":         [5, 5, 4, 2, 4, 2],
    "moslashuvchanlik": [5, 4, 3, 2, 4, 5],
    "quvvat_vt":        [510, 850, 200, 5.5, 60, 325],
    "oflayn":           [1, 1, 0, 1, 1, 1],
}, index=["CPU", "GPU", "TPU", "NPU", "Jetson", "FPGA"])
print("=== Mezonlar jadvali (oldingi qadamlardan, taxminiy) ===")
print(JADVAL.to_string())

SSENARIY = {  # vaznlar yig'indisi 1; cheklov — lambda
    "Data-markaz: katta hajmli inferens": ({"unumdorlik": .30, "kechikish": .10, "energiya":
.15, "TCO": .25, "ekotizim": .15, "moslashuvchanlik": .05}, lambda r: True),
    "Chekka kamera: <= 60 Vt, internetsiz":  ({"unumdorlik": .10, "kechikish": .20, "energiya":
.35, "TCO": .15, "ekotizim": .10, "moslashuvchanlik": .10}, lambda r: r.quvvat_vt <= 60 and
r.oflayn == 1),
```

```
    "Moliya: past kechikish, tez o'zgaruvchi model": ({"unumdorlik": .10, "kechikish": .35,
"energiya": .05, "TCO": .10, "ekotizim": .15, "moslashuvchanlik": .25}, lambda r: r.oflayn ==
1),
}


def normal(df):
    """Min-max normallash: 1 — eng yaxshi, 0 — eng yomon (yo'nalish hisobga olinadi). Log-
shkala — katta diapazonlar uchun."""
    n = pd.DataFrame(index=df.index)
    for m, yo in MEZON.items():
        x = np.log10(df[m].astype(float)) if m in ("unumdorlik", "energiya", "TCO") else
df[m].astype(float)
        z = (x - x.min()) / (x.max() - x.min() + 1e-12)
        n[m] = z if yo > 0 else 1 - z
    return n


def topsis(df, w):
    """TOPSIS: ideal va anti-ideal yechimgacha masofa nisbati."""
    X = df[list(MEZON)].astype(float).values
    X = X / np.sqrt((X ** 2).sum(0)); V = X * np.array([w[m] for m in MEZON])
    yo = np.array(list(MEZON.values()))
    ideal = np.where(yo > 0, V.max(0), V.min(0)); anti = np.where(yo > 0, V.min(0), V.max(0))
    dp = np.sqrt(((V - ideal) ** 2).sum(1)); dm = np.sqrt(((V - anti) ** 2).sum(1))
    return pd.Series(dm / (dp + dm), index=df.index)


issiq = {}
for nom, (w, cheklov) in SSENARIY.items():
    nomzod = JADVAL[JADVAL.apply(cheklov, axis=1)]
    N = normal(nomzod)
    wsm = (N * pd.Series(w)).sum(axis=1)
    tp = topsis(nomzod, w)
    # Monte-Karlo: vaznlarni Dirichlet bilan +-tebratamiz (konsentratsiya 50) — 1-o'rin
barqarorligi
    birinchi = pd.Series(0, index=nomzod.index)
    W0 = np.array([w[m] for m in MEZON])
    for _ in range(2000):
        wv = rng.dirichlet(W0 * 50)
        birinchi.iloc[int((N.values * wv).sum(1).argmax())] += 1
    res = pd.DataFrame({"WSM": wsm, "TOPSIS": tp, "1-o'rin_%": 100 * birinchi /
2000}).sort_values("WSM", ascending=False)
    res["o'rin"] = range(1, len(res) + 1)
    print(f"\n=== Ssenariy: {nom} ===")
    print(f"Chiqarib tashlangan (cheklov): {', '.join(sorted(set(JADVAL.index) -
set(nomzod.index))) or "yo'q"}")
    print(res.to_string(float_format=lambda v: f"{v:.3f}"))
    print(f"Tavsiya: {res.index[0]} (WSM), TOPSIS bo'yicha: {res.TOPSIS.idxmax()}")
    if res.index[0] != res.TOPSIS.idxmax():
        print("Usullar kelishmadi: nomzod kam bo'lsa min-max normallash har mezonni 0/1 ga
aylantiradi — qaror chegarada.")
    issiq[nom.split(":")[0]] = wsm.reindex(JADVAL.index)

print("\n=== Yakuniy tanlov matritsasi (WSM, '—' = cheklov bo'yicha mos emas) ===")
M = pd.DataFrame(issiq)
print(M.to_string(float_format=lambda v: f"{v:.2f}", na_rep="—"))

fig, ax = plt.subplots(figsize=(7, 4))
im = ax.imshow(M.fillna(0).values, cmap="viridis", aspect="auto")
ax.set_xticks(range(M.shape[1])); ax.set_xticklabels(M.columns, fontsize=8)
ax.set_yticks(range(M.shape[0])); ax.set_yticklabels(M.index)
for i in range(M.shape[0]):
```

```
    for j in range(M.shape[1]):
        v = M.values[i, j]; ax.text(j, i, "—" if np.isnan(v) else f"{v:.2f}", ha="center",
va="center", color="w", fontsize=8)
ax.set_title("Tanlov matritsasi (WSM ball)"); plt.colorbar(im); plt.tight_layout()
print("\n=== Tanlov matritsasi tayyor ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Mezonlar jadvali`, `=== Ssenariy:`, `=== Yakuniy tanlov matritsasi`, `=== Tanlov matritsasi tayyor ===`

TAJRIBA QILIB KO‘RING

- Data-markaz ssenariysiga 'quvvat_vt >= 100' (faqat server qurilmalari) cheklovini qo'shing va reytingni qayta ko'ring.
- Ekotizim va moslashuvchanlik ekspert baholarini o'zgartiring — tavsiya qachon o'zgaradi?
- To'rtinchi ssenariy (LLM dekod xizmati) qo'shing: xotira o'tkazuvchanligi mezoni bilan.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Pasport jadvalini tekshirish** 1-qadam jadvalidagi har qurilma uchun cho'qqi INT8/FP16 qiymati, xotira o'tkazuvchanligi va quvvatni ishlab chiqaruvchining rasmiy ma'lumot varag'i (datasheet) bilan solishtiring; farq bo'lsa tuzating va manbalar ro'yxatini (havola yoki hujjat nomi, sana) jadval ostida bering. Ridge nuqtalarini qayta hisoblang.	oson	5
2	**O'z yuklamangizning AI si** 1-qadamga o'zingiz tanlagan modelning 3 ta qatlamini (masalan, YOLO conv, Transformer attention QK^T, embedding qidiruv) qo'shing: amallar va baytlar formulasini yozing, AI ni hisoblang va har arxitekturada chegara turini (H/X) jadvalda ko'rsating.	oson	5
3	**Sistolik massiv to'g'riligi va to'lqin fronti** 2-qadam simulyatorini 12x12 massiv va K=12, 48, 192 uchun ishga tushiring, taktlar formulasini tasdiqlang va t=5, 11, 17 dagi faol PE xaritalarini chiqaring. PE foydalanishning K ga bog'liqligi grafigini topshiring.	oson	5
4	**MXU uchun padding strategiyasi** 2-qadam modelida N = 100...300 (qadam 10) uchun MXU foydalanishini hisoblang, 128 ga karrali bo'lmagan o'lchamlardagi 'arra tishi' grafigini chizing. Toq o'lchamli qatlam uchun 3 strategiyani (padding, o'lchamni qisqartirish, ikki qatlamni birlashtirish) samarali TFLOP/s bo'yicha solishtiring.	o'rta	10
5	**FPGA fixed-point dizaynini tanlash** 3-qadamdagi MLP uchun har qatlamga alohida (og'irlik, faollashtirish, akkumulyator) bit kengligini tanlang: aniqlik pasayishi <= 0,5 p.p. bo'lgan eng tejamkor konfiguratsiyani toping. DSP soni va o'tkazuvchanlikni (300 MHz, II=1) modelda baholang va jadvalda bering.	o'rta	10
6	**Kalibrlash va observer tadqiqoti** 4-qadamda MinMax, MovingAverageMinMax va Histogram observerlarini 4, 16, 64, 256 namunali kalibrlash bilan solishtiring (aniqlik, kalibrlash vaqti). Natijani 4x3 jadvalda va qisqa xulosada bering: NPU kompilyatoriga qaysi kalibrlash to'plamini berardingiz?	o'rta	10
7	**Sandbox CPU Roofline'ini aniqlashtirish** 5-qadamda triad o'lchovini 1, 4, 16, 64 MB massivlar bilan takrorlang (kesh ierarxiyasi ko'rinsin) va GEMM'ni 1 va 2 ip bilan o'lchang. Kesh darajalari uchun alohida 'shift' chiziqli Roofline grafigini chizing va GEMV ning 100 % dan yuqori samaradorligini tushuntiring.	o'rta	10
8	**TCO modeliga o'z mintaqangiz narxlari** 6-qadam modelini O'zbekiston sharoitiga moslang: elektr narxi (amaldagi tarif, manba bilan), import bojxona xarajatlari, sovutish (PUE) va xizmat ko'rsatish (yillik 10 % CAPEX) qo'shing. 3 ta talab darajasi (500, 5000, 50000 tasvir/s) uchun TCO va 1 mln tasvir narxini hisoblang; natija qaysi arxitekturani tanlashga ta'sir qilishini tahlil qiling. Barcha farazlarni taxminiy deb belgilang.	qiyin	15

9	**Ekstrapolyatsiyani tekshirish** 5-qadam usuli bilan ONNX Runtime'da kichik CNN (yoki digits MLP) inferensini sandbox CPU'da o'lchang, so'ng uni qatlam bo'yicha Roofline modelida jamlab boshqa arxitekturalar uchun bashorat qiling. Bashoratni ochiq manbadagi (MLPerf Inference yoki ishlab chiqaruvchi benchmark) mos model natijasi bilan solishtiring va xato manbalarini (ishga tushirish xarajati, fusion, kesh) tahlil qiling.	qiyin	15
10	**To'liq qiyosiy hisobot va tanlov matritsasi** O'z AI ilovangiz (masalan, shahar kameralari, tibbiy tasvir, chatbot) uchun 1-7 qadamlarni moslab, CPU/GPU/TPU/NPU/FPGA bo'yicha yakuniy hisobot tayyorlang: pasport, Roofline, kvantlash ta'siri, energiya va TCO, kamida 3 ssenariyli tanlov matritsasi va vaznlar sezgirligi. Tavsiyani raqamlar bilan asoslang (kamida 400 so'z), barcha taxminiy qiymatlarni alohida belgilang.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Tibbiyot: KT tasvirlarida o'smani segmentlash serveri oson

Soha: Tibbiyot (radiologiya)

Viloyat diagnostika markazi KT kesimlarini U-Net bilan segmentlovchi server sotib olmoqchi. Kuniga ~3000 kesim, har biri 512x512, natija 2 daqiqa ichida kerak. Bemor ma'lumoti binodan chiqmasligi sababli bulut faqat taqqoslash uchun ko'riladi. Siz CPU, GPU va FPGA ni Roofline, energiya va TCO bo'yicha solishtirib, tavsiya berasiz.

Yuklama/model	U-Net conv3x3 512x512x64->64 va 1x1 qatlam
Talab	3000 kesim/kun, javob <= 120 s
Cheklov	bulutsiz, 1 kVt gacha
Solishtiriladi	CPU vs GPU vs FPGA
Fixed-point format	Q2.5 / Q4.11
Narx farazi	elektr 0,08 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (U-Net conv3x3 512x512x64->64 va 1x1 qatlam) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (Q2.5 / Q4.11) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (3000 kesim/kun, javob <= 120 s) va cheklov (bulutsiz, 1 kVt gacha) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (elektr 0,08 $/kVt*soat); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

2-variant. Agro: dron tasvirlarida begona o'tni aniqlash oson

Soha: Qishloq xo'jaligi

Paxta dalasida dron 30 kadr/s 1280x720 tasvir oladi, begona o'tlar real vaqtda aniqlanib purkagichga signal beriladi. Dron batareyasi chekli: hisoblash moduli 15 Vt dan oshmasligi kerak. Hailo-8 kabi NPU va Jetson sinfidagi SoC bilan bort CPU ni solishtirasiz.

Yuklama/model	YOLO uslubidagi depthwise conv + 1x1
Talab	30 kadr/s, kechikish <= 50 ms
Cheklov	<= 15 Vt, internetsiz
Solishtiriladi	CPU vs NPU vs Jetson
Fixed-point format	Q1.6 / Q3.4
Narx farazi	batareya 100 Vt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs NPU vs Jetson + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (YOLO uslubidagi depthwise conv + 1x1) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (Q1.6 / Q3.4) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (30 kadr/s, kechikish <= 50 ms) va cheklov (<= 15 Vt, internetsiz) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (batareya 100 Vt*soat); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

3-variant. Transport: avtomobil raqamini tanish (ANPR) shlyuzi oson

Soha: Transport

Shahar chorrahalarida 16 ta kamera, har biridan 10 kadr/s; raqam aniqlash va OCR mahalliy shlyuzda bajariladi. Shlyuz shkafda turadi, sovutish passiv, 60 Vt gacha. Arxitekturalarni kadr/s/Vt va 5 yillik TCO bo'yicha baholaysiz.

Yuklama/model	conv3x3 + kichik LSTM/GEMM (OCR)
Talab	160 kadr/s
Cheklov	<= 60 Vt, passiv sovutish
Solishtiriladi	CPU vs Jetson vs FPGA
Fixed-point format	Q3.4
Narx farazi	5 yil, 0,10 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs Jetson vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (conv3x3 + kichik LSTM/GEMM (OCR)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (Q3.4) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (160 kadr/s) va cheklov (<= 60 Vt, passiv sovutish) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (5 yil, 0,10 $/kVt*soat); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

4-variant. Telekom: tarmoq trafigidagi anomaliyalarni aniqlash oson

Soha: Telekom

Operator yadro tarmog'ida har soniyada 2 mln paket metadatasi avtoenkoder bilan tekshiriladi. Kechikish 1 ms dan oshsa paket navbati to'lib ketadi. FPGA (SmartNIC) bilan GPU va CPU ni GEMV-og'ir yuklamada solishtirasiz.

Yuklama/model	MLP avtoenkoder 64-32-8-32-64 (GEMV, batch kichik)
Talab	2 mln namuna/s, p99 <= 1 ms
Cheklov	server shkafi, 300 Vt
Solishtiriladi	CPU vs GPU vs FPGA
Fixed-point format	Q1.14 (16 bit)
Narx farazi	elektr 0,12 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (MLP avtoenkoder 64-32-8-32-64 (GEMV, batch kichik)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (Q1.14 (16 bit)) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (2 mln namuna/s, p99 <= 1 ms) va cheklov (server shkafi, 300 Vt) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (elektr 0,12 $/kVt*soat); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

5-variant. Energetika: quyosh paneli nuqsonlarini termal tasvirda topish oson

Soha: Energetika

Quyosh elektr stansiyasida 40 ming panel, dron termal kamerasi bilan haftasiga bir marta suratga olinadi (~200 ming tasvir). Ishlov kechasi paketli bajariladi, kechikish muhim emas, narx muhim. Bulut TPU, egalikdagi GPU va CPU klasterini solishtirasiz.

Yuklama/model	ResNet-18 sinfidagi tasnif (1,8 GMAC)
Talab	200 ming tasvir 8 soatda
Cheklov	kechikish muhim emas
Solishtiriladi	CPU vs GPU vs TPU
Fixed-point format	BF16
Narx farazi	bulut TPU soatbay

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs TPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (ResNet-18 sinfidagi tasnif (1,8 GMAC)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (BF16) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (200 ming tasvir 8 soatda) va cheklov (kechikish muhim emas) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (bulut TPU soatbay); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

6-variant. Moliya: kartadagi firibgarlikni real vaqtda baholash oson

Soha: Moliya (bank)

Bank protsessing markazida har tranzaksiya 5 ms ichida gradient boosting + kichik MLP bilan baholanishi kerak; cho'qqida 8000 tranzaksiya/s. Ishonchlilik uchun ikki ma'lumot markazi. CPU, GPU va FPGA ni p99 kechikish va TCO bo'yicha solishtirasiz.

Yuklama/model	MLP 256-128-1 + daraxtlar (batch 1)
Talab	8000 tranz./s, p99 <= 5 ms
Cheklov	2 DC, zaxira 2N
Solishtiriladi	CPU vs GPU vs FPGA
Fixed-point format	Q3.12
Narx farazi	3 yil, 2 ta DC

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (MLP 256-128-1 + daraxtlar (batch 1)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (Q3.12) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (8000 tranz./s, p99 <= 5 ms) va cheklov (2 DC, zaxira 2N) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (3 yil, 2 ta DC); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

7-variant. Ta'lim: imtihon videonazorati (proktoring) xizmati oson

Soha: Ta'lim

Universitet onlayn imtihonida 2000 talaba bir vaqtda, har biridan 2 kadr/s yuz va ko'z harakati tahlili qilinadi. Imtihonlar yiliga 30 kun, qolgan vaqt yuklama past. Egalikdagi GPU serveri va bulut GPU/TPU ni yuklama koeffitsienti bo'yicha solishtirasiz.

Yuklama/model	MobileNetV2 (0,3 GMAC) + yuz nuqtalari
Talab	4000 kadr/s, 30 kun/yil
Cheklov	mavsumiy yuklama
Solishtiriladi	GPU egalik vs bulut GPU vs TPU
Fixed-point format	INT8
Narx farazi	yuklama koeffitsienti 8 %

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (GPU egalik vs bulut GPU vs TPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (MobileNetV2 (0,3 GMAC) + yuz nuqtalari) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (INT8) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (4000 kadr/s, 30 kun/yil) va cheklov (mavsumiy yuklama) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (yuklama koeffitsienti 8 %); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

8-variant. Sanoat: konveyerda metall quyma nuqsonini aniqlash o'rta

Soha: Sanoat (metallurgiya)

Quyish sexida konveyer 2 m/s tezlikda, 4 kamera 120 kadr/s. Nuqsonli detal 20 ms ichida ajratilishi kerak, sex harorati yuqori va chang ko'p. FPGA (deterministik kechikish) va Jetson ni CPU bilan solishtirasiz.

Yuklama/model	conv3x3 256x256x32->32, 5 qatlam
Talab	480 kadr/s, kechikish <= 20 ms (qat'iy)
Cheklov	IP65 shkaf, 100 Vt
Solishtiriladi	CPU vs Jetson vs FPGA
MXU/sistolik holat	N=32
Fixed-point format	Q2.5 / Q2.13
Narx farazi	elektr 0,07 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs Jetson vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (conv3x3 256x256x32->32, 5 qatlam) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (N=32) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q2.5 / Q2.13) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (480 kadr/s, kechikish <= 20 ms (qat'iy)) va cheklov (IP65 shkaf, 100 Vt) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (elektr 0,07 $/kVt*soat); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

9-variant. Xavfsizlik: aeroportda bagaj rentgen tasvirini skrining o'rta

Soha: Xavfsizlik

Aeroport bagaj liniyasida soatiga 1800 sumka, har biri 4 proyeksiyali rentgen tasvir. Taqiqlangan buyumni aniqlash detektori 1 s ichida operatorga ko'rsatishi kerak. GPU serveri, FPGA va CPU ni ishonchlilik va TCO bo'yicha taqqoslaysiz.

Yuklama/model	Faster R-CNN sinfidagi detektor (~90 GMAC)
Talab	2 tasvir/s, javob <= 1 s
Cheklov	ishonchlilik 99,99 %
Solishtiriladi	CPU vs GPU vs FPGA
MXU/sistolik holat	M=4
Fixed-point format	Q4.11
Narx farazi	7 yil xizmat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (Faster R-CNN sinfidagi detektor (~90 GMAC)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=4) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q4.11) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (2 tasvir/s, javob <= 1 s) va cheklov (ishonchlilik 99,99 %) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (7 yil xizmat); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

10-variant. Chakana savdo: do'kon javonlarini monitoring qilish o'rta

Soha: Chakana savdo

Supermarketlar tarmog'i 120 do'konda javonlardagi bo'sh joylarni har 5 daqiqada tekshiradi; har do'konda 20 kamera. Variant: har do'konda chekka qurilma yoki markaziy GPU serveri. Energiya va trafik narxi bilan 3 yillik TCO hisoblaysiz.

Yuklama/model	EfficientDet-D0 (2,5 GMAC)
Talab	120 do'kon x 20 kamera / 5 daq
Cheklov	do'konda 20 Vt gacha
Solishtiriladi	NPU (har do'kon) vs markaziy GPU vs CPU
MXU/sistolik holat	M=8
Fixed-point format	INT8 per-channel
Narx farazi	trafik 0,05 $/GB

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (NPU (har do'kon) vs markaziy GPU vs CPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (EfficientDet-D0 (2,5 GMAC)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=8) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (INT8 per-channel) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (120 do'kon x 20 kamera / 5 daq) va cheklov (do'konda 20 Vt gacha) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (trafik 0,05 $/GB); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

11-variant. Logistika: ombor robotlari uchun yo'l rejalashtirish o'rta

Soha: Logistika

Avtomatlashtirilgan omborda 150 robot, har biri 10 Gs chastotada lidar nuqtalar bulutini kichik PointNet bilan tahlil qiladi. Hisoblash robot bortida (Jetson) yoki markaziy serverda (Wi-Fi kechikishi 15 ms) bo'lishi mumkin. Variantlarni kechikish va energiya bo'yicha solishtirasiz.

Yuklama/model	PointNet: GEMM 1024x64->128 va max-pool
Talab	150 robot x 10 Gs
Cheklov	bort batareyasi, 25 Vt
Solishtiriladi	Jetson (bort) vs GPU (markaz) vs CPU
MXU/sistolik holat	M=1024
Fixed-point format	FP16
Narx farazi	Wi-Fi RTT 15 ms

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (Jetson (bort) vs GPU (markaz) vs CPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (PointNet: GEMM 1024x64->128 va max-pool) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=1024) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (FP16) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (150 robot x 10 Gs) va cheklov (bort batareyasi, 25 Vt) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (Wi-Fi RTT 15 ms); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

12-variant. Shahar: aqlli svetofor uchun transport oqimini bashorat qilish o'rta

Soha: Shahar infratuzilmasi

Shahar 300 chorrahasida har 10 s da GNN/LSTM bilan keyingi 5 daqiqa oqimi bashorat qilinadi. Hisob markaziy data-markazda, kechikish 2 s gacha. GEMV-og'ir LSTM yuklamada xotira o'tkazuvchanligi hal qiluvchi ekanini Roofline bilan ko'rsatasiz.

Yuklama/model	LSTM 512 yashirin, 300 ketma-ketlik
Talab	30 bashorat/s, kechikish <= 2 s
Cheklov	data-markaz
Solishtiriladi	CPU vs GPU vs TPU
MXU/sistolik holat	M=300
Fixed-point format	BF16
Narx farazi	PUE 1,6

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs TPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (LSTM 512 yashirin, 300 ketma-ketlik) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=300) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (BF16) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (30 bashorat/s, kechikish <= 2 s) va cheklov (data-markaz) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (PUE 1,6); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

13-variant. Ekologiya: havo sifati sensorlarida chekka tahlil o'rta

Soha: Ekologiya

Shahar bo'ylab 500 ta havo sifati stansiyasi quyosh paneli bilan ishlaydi (o'rtacha 5 Vt). Har stansiya har daqiqada 1D-CNN bilan ifloslanish manbasini tasniflaydi. Mikro-NPU, kichik CPU (Raspberry Pi sinfi) va bulutga yuborishni energiya bo'yicha solishtirasiz.

Yuklama/model	1D-CNN 6 qatlam, 1440 nuqta
Talab	1 tasnif/daq/stansiya
Cheklov	<= 5 Vt, quyosh
Solishtiriladi	CPU (RPi) vs NPU vs bulut
MXU/sistolik holat	N=24
Fixed-point format	Q1.6 (8 bit)
Narx farazi	LTE trafik 2 $/GB

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU (RPi) vs NPU vs bulut + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (1D-CNN 6 qatlam, 1440 nuqta) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (N=24) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q1.6 (8 bit)) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (1 tasnif/daq/stansiya) va cheklov (<= 5 Vt, quyosh) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (LTE trafik 2 $/GB); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

14-variant. Sport: futbol o'yinida o'yinchilarni kuzatish o'rta

Soha: Sport

Stadionda 8 kamera 50 kadr/s 4K oqim beradi, o'yinchi va to'p koordinatalari 100 ms kechikish bilan efirga chiqariladi. O'yinlar haftasiga 2 marta, 2 soatdan. Mobil GPU serveri va bulut GPU ni foydalanish koeffitsienti bilan baholaysiz.

Yuklama/model	YOLOv8-m sinfidagi detektor (~40 GMAC)
Talab	400 kadr/s, <= 100 ms
Cheklov	mobil stend, 2 kVt
Solishtiriladi	GPU egalik vs bulut GPU vs FPGA
MXU/sistolik holat	M=8
Fixed-point format	FP16 / INT8
Narx farazi	haftada 4 soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (GPU egalik vs bulut GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (YOLOv8-m sinfidagi detektor (~40 GMAC)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=8) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (FP16 / INT8) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (400 kadr/s, <= 100 ms) va cheklov (mobil stend, 2 kVt) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (haftada 4 soat); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

15-variant. Media: video kontentni avtomatik subtitrlash o'rta

Soha: Media

Teleradiokompaniya arxivida 20 ming soat video; Whisper sinfidagi model bilan o'zbek tilida subtitr yaratiladi. Kodlovchi qism hisobga, dekoder xotiraga og'ir. Bulut TPU, GPU va CPU klasterida arxivni 30 kunda qayta ishlash narxini solishtirasiz.

Yuklama/model	Transformer kodlovchi (GEMM 1500x512x2048) + dekoder GEMV
Talab	20 ming soat / 30 kun
Cheklov	faqat narx va muddat
Solishtiriladi	CPU vs GPU vs TPU
MXU/sistolik holat	M=1500 (kodlovchi), M=1 (dekoder)
Fixed-point format	BF16
Narx farazi	bulut soatbay

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs TPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (Transformer kodlovchi (GEMM 1500x512x2048) + dekoder GEMV) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=1500 (kodlovchi), M=1 (dekoder)) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (BF16) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (20 ming soat / 30 kun) va cheklov (faqat narx va muddat) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (bulut soatbay); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

16-variant. Tibbiyot: EKG signalidan aritmiyani taqiladigan qurilmada aniqlash

qiyin

Soha: Tibbiyot (kardiologiya)

Taqiladigan EKG qurilmasi 250 Gs signalni 1D-CNN bilan har 10 s da tahlil qiladi, batareya 7 kun yetishi shart (400 mAh, 3,7 V). Mikro-NPU, FPGA (past quvvatli) va MCU-CPU uchun energiya byudjeti va fixed-point aniqlikni hisoblaysiz.

Yuklama/model	1D-CNN 8 qatlam, 2500 nuqta
Talab	1 tahlil/10 s
Cheklov	batareya 7 kun, < 10 mVt o'rtacha
Solishtiriladi	CPU (MCU) vs NPU vs FPGA
MXU/sistolik holat	N=16
Fixed-point format	Q1.6 / Q1.14
Narx farazi	akkumulyator 1,48 Vt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU (MCU) vs NPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (1D-CNN 8 qatlam, 2500 nuqta) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (N=16) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q1.6 / Q1.14) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (1 tahlil/10 s) va cheklov (batareya 7 kun, < 10 mVt o'rtacha) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (akkumulyator 1,48 Vt*soat); sezgirlik tahlili.
7. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan); Manbalar ro'yxati (datasheet, MLPerf) va farazlar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

17-variant. Neft-gaz: quvurda oqishni akustik signal bo'yicha aniqlash qiyin

Soha: Neft-gaz

Magistral quvur bo'ylab har 5 km da akustik sensor, 48 kGs signal spektrogrammasi CNN bilan tahlil qilinadi. Stansiyalar uzoqda, quvvat quyosh+akkumulyator (30 Vt), aloqa sun'iy yo'ldosh orqali qimmat. Chekkada FPGA va Jetson ni solishtirasiz.

Yuklama/model	spektrogramma CNN 128x128, 4 qatlam
Talab	har stansiya 2 tahlil/s
Cheklov	<= 30 Vt, sun'iy yo'ldosh aloqasi
Solishtiriladi	Jetson vs FPGA vs NPU
MXU/sistolik holat	N=64
Fixed-point format	Q3.12
Narx farazi	aloqa 5 $/MB

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (Jetson vs FPGA vs NPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (spektrogramma CNN 128x128, 4 qatlam) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (N=64) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q3.12) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (har stansiya 2 tahlil/s) va cheklov (<= 30 Vt, sun'iy yo'ldosh aloqasi) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (aloqa 5 $/MB); sezgirlik tahlili.
7. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan); Manbalar ro'yxati (datasheet, MLPerf) va farazlar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

18-variant. Bank: chatbot uchun LLM inferens klasteri qiyin

Soha: Moliya (mijozlarga xizmat)

Bank mijozlarga 7B parametrli LLM asosidagi chatbotni ichki serverda joylashtirmoqchi: cho'qqida 200 parallel suhbat, javob tezligi kamida 15 token/s. Dekod bosqichi xotira o'tkazuvchanligi bilan cheklangani sababli HBM'li GPU, TPU va CPU ni token/s/$ bo'yicha solishtirasiz.

Yuklama/model	LLM 7B: dekod GEMV 4096x4096, prefill GEMM
Talab	200 suhbat x 15 token/s
Cheklov	ma'lumot ichkarida
Solishtiriladi	CPU vs GPU vs TPU
MXU/sistolik holat	dekod M=1..64
Fixed-point format	INT8 og'irlik / FP16
Narx farazi	3 yil, 0,10 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs TPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (LLM 7B: dekod GEMV 4096x4096, prefill GEMM) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (dekod M=1..64) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (INT8 og'irlik / FP16) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (200 suhbat x 15 token/s) va cheklov (ma'lumot ichkarida) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (3 yil, 0,10 $/kVt*soat); sezgirlik tahlili.
7. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan); Manbalar ro'yxati (datasheet, MLPerf) va farazlar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

19-variant. Kosmik monitoring: sun'iy yo'ldosh tasvirlarida ekin turlarini tasniflash qiyin

Soha: Masofaviy zondlash

Davlat agentligi har 5 kunda respublika hududining 10 m aniqlikdagi multispektral tasvirlarini (13 kanal) yangilaydi: ~45 ming kvadrat 256x256. Tasniflash 12 soatda tugashi kerak. Egalikdagi GPU, bulut TPU va FPGA ni katta batch rejimida solishtirasiz.

Yuklama/model	U-Net yengil, 13 kanal kirish
Talab	45 ming plitka / 12 soat
Cheklov	paketli, kechikish muhim emas
Solishtiriladi	GPU vs TPU vs FPGA
MXU/sistolik holat	toq K=13*9=117
Fixed-point format	BF16 / INT8
Narx farazi	5 kunda 1 ishga tushirish

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (GPU vs TPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (U-Net yengil, 13 kanal kirish) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (toq K=13*9=117) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (BF16 / INT8) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (45 ming plitka / 12 soat) va cheklov (paketli, kechikish muhim emas) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (5 kunda 1 ishga tushirish); sezgirlik tahlili.
7. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan); Manbalar ro'yxati (datasheet, MLPerf) va farazlar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

20-variant. Kiberxavfsizlik: zararli fayllarni bayt-ketma-ketlik bo'yicha aniqlash

qiyin

Soha: Kiberxavfsizlik

Korporativ pochta shlyuzida kuniga 1,5 mln ilova MalConv uslubidagi 1D-CNN bilan tekshiriladi, cho'qqi 60 fayl/s, kechikish 300 ms. Shlyuz 1U serverga sig'ishi kerak (bitta PCIe slot). GPU, FPGA va CPU ni slot, quvvat va TCO bo'yicha solishtirasiz.

Yuklama/model	1D-CNN embedding 8 + conv 512 filtr, 2 MB kirish
Talab	60 fayl/s, p95 <= 300 ms
Cheklov	1U, bitta PCIe, 250 Vt
Solishtiriladi	CPU vs GPU vs FPGA
MXU/sistolik holat	M=1..16
Fixed-point format	Q3.4 (8 bit)
Narx farazi	elektr 0,10 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (1D-CNN embedding 8 + conv 512 filtr, 2 MB kirish) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=1..16) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q3.4 (8 bit)) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (60 fayl/s, p95 <= 300 ms) va cheklov (1U, bitta PCIe, 250 Vt) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (elektr 0,10 $/kVt*soat); sezgirlik tahlili.
7. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan); Manbalar ro'yxati (datasheet, MLPerf) va farazlar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20